Cross-Encoders and Re-Ranking

A Detailed Guide to Cross-Encoder Architecture and Two-Stage Retrieval

Prepared for Sourav Bhattacharya

Created: July 7, 2026, 15:10 UTC

Contents

1. What Is a Cross-Encoder? 3
- 1.1 Why the Distinction Matters . . . 3

2. Bi-Encoder vs. Cross-Encoder Architecture 4
- 2.1 Bi-Encoder: Two Independent Towers . . . 4
- 2.2 Cross-Encoder: One Joint Pass . . . 5
- 2.3 Side-by-Side Comparison . . . 5

3. Re-Ranking: Why a Second Stage Is Needed 7
- 3.1 The Two-Stage Pipeline . . . 7
- 3.2 A Worked Example . . . 7

4. Using Cross-Encoders in Python 8
- 4.1 Installation . . . 8
- 4.2 Scoring Query-Document Pairs Directly . . . 8
- 4.3 Convenience Method: `rank()` . . . 8
- 4.4 Full Retrieve-and-Rerank Pipeline . . . 9

5. Popular Pretrained Cross-Encoder / Reranker Models 11

6. Evaluating a Reranker 12

7. Training a Custom Cross-Encoder 13
- 7.1 Loss Functions . . . 13
- 7.2 Minimal Training Example . . . 13

8. Trade-offs and Best Practices 14
- 8.1 Latency vs. Precision . . . 14
- 8.2 Re-Ranking Is a Precision Layer, Not a Recall Fix . . . 14
- 8.3 Batch and Cache Where Possible . . . 14
- 8.4 Choosing `k` . . . 14

9. Summary 15

References **16**

1. What Is a Cross-Encoder?

A cross-encoder is a transformer model that takes **two pieces of text at once** — typically a query and a candidate document — concatenates them into a single input sequence, and passes that combined sequence through one shared encoder. Because both texts are present in the same forward pass, every token of the query can attend to every token of the document (and vice versa) through the transformer's self-attention layers. The model then produces a single number: a relevance score describing how well the document answers or matches the query.

This is different from the more familiar **bi-encoder** (also called a "sentence embedding model" or "dual encoder"), which encodes the query and the document *separately* into two vectors and then compares those vectors with a cheap operation like cosine similarity. Bi-encoders are what power most vector databases and semantic search systems. Cross-encoders are what make **re-ranking** — the second, precision-focused pass over a shortlist of candidates — so effective.

1.1 Why the Distinction Matters

Search and retrieval-augmented generation (RAG) systems face a tension between two goals:

1. **Speed and scale** — a system may need to search millions or billions of documents in milliseconds.
2. **Precision** — the top few results shown to a user (or fed to an LLM) need to actually be the most relevant ones.

Bi-encoders solve (1) well because document vectors can be computed once, offline, and stored in an approximate-nearest-neighbor (ANN) index. Comparing a query vector against that index is extremely fast, even at huge scale. But because the query and document never "see" each other during encoding, bi-encoders can miss subtle relevance signals — for example, negation, precise numeric matching, or a document that uses different wording for the same concept.

Cross-encoders solve (2) well because full attention between query and document tokens lets the model reason about nuanced word-level relationships. The trade-off is that a cross-encoder must run a full transformer forward pass for *every single* query-document pair — there is no way to pre-compute a document representation in isolation, because the model has no notion of a document's meaning independent of whatever query it is paired with.

This is why, in practice, the two architectures are used **together**, not as competitors: a bi-encoder narrows millions of documents down to a shortlist, and a cross-encoder re-ranks that shortlist with much higher precision.

2. Bi-Encoder vs. Cross-Encoder Architecture

2.1 Bi-Encoder: Two Independent Towers

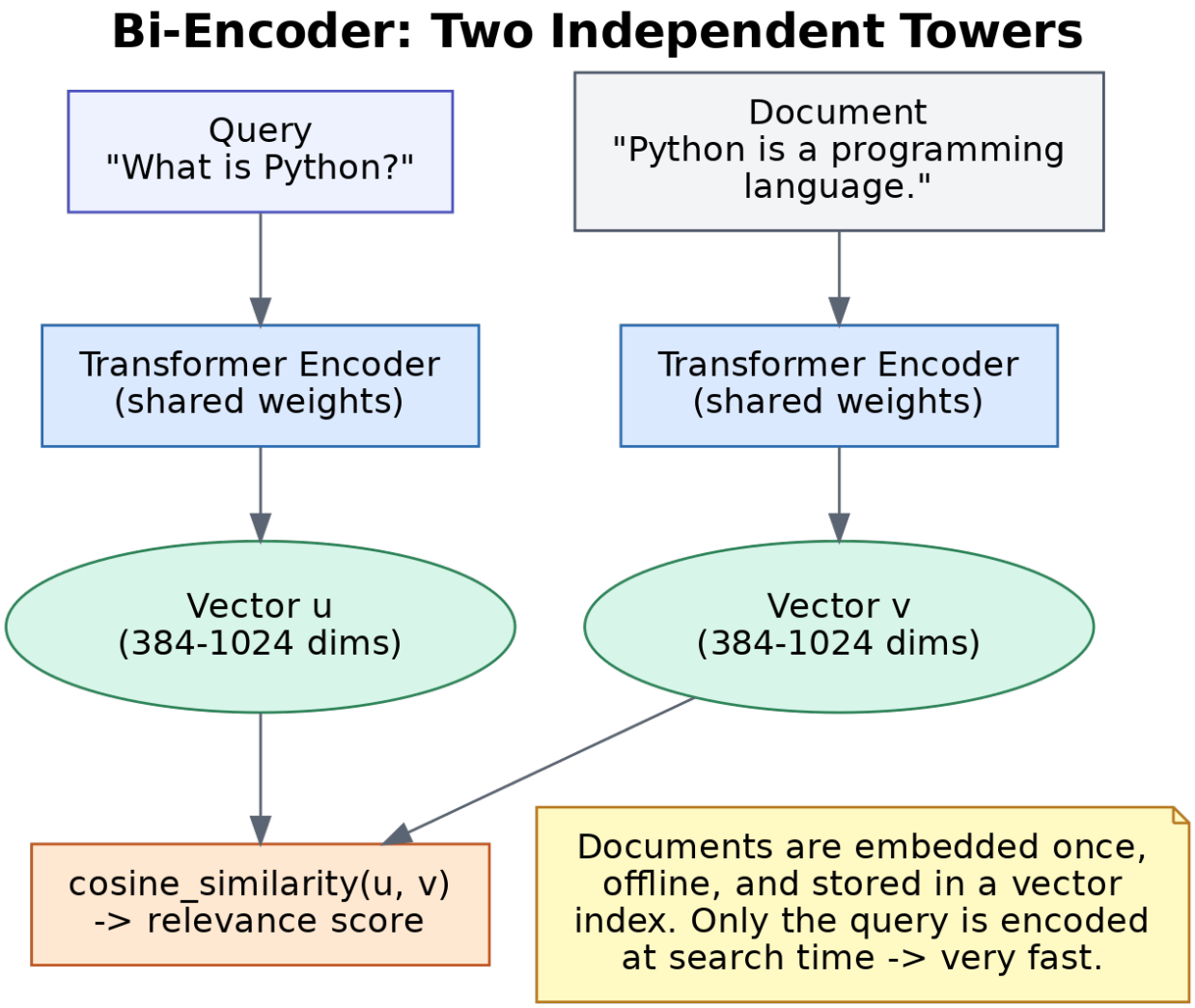

Figure 1: The bi-encoder encodes the query and document in separate passes and compares the resulting vectors with cosine similarity.

The query and the document are each fed through the *same* transformer encoder (shared weights), but never together. Each produces a fixed-size vector — commonly 384 to 1024 dimensions. Similarity between a query vector u and a document vector v is computed with a cheap operation such as cosine similarity or dot product.

Because the document side of this computation does not depend on the query at all, document embeddings can be computed **once, offline**, and stored in a vector index (FAISS, HNSW, Qdrant, and similar). At query time, only the query needs to be encoded, and the ANN index handles the search over millions of stored vectors in milliseconds.

2.2 Cross-Encoder: One Joint Pass

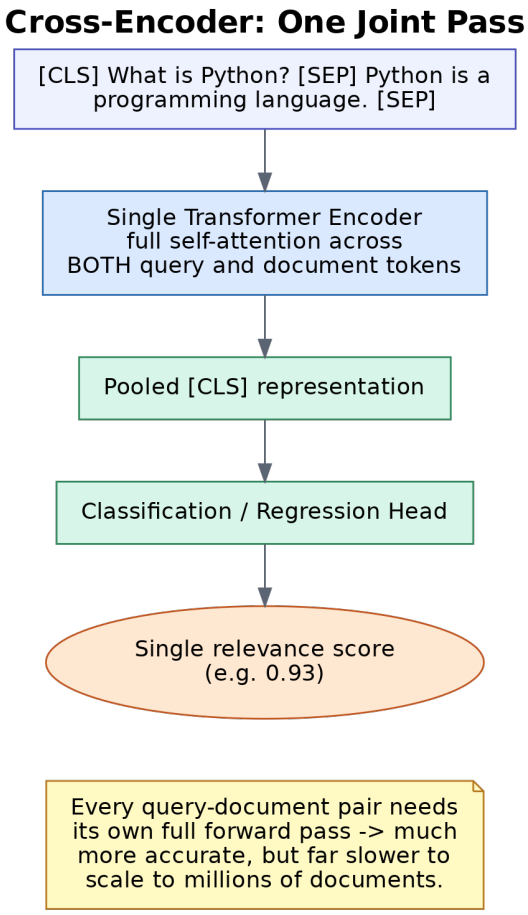

Figure 2: The cross-encoder concatenates the query and document and encodes them together, producing a single relevance score.

The query and document are concatenated into one sequence, typically formatted as:

`[CLS] query text [SEP] document text [SEP]`

This combined sequence is passed through a single transformer. Self-attention lets every query token attend directly to every document token within the same set of layers, which is what gives cross-encoders their precision advantage. The pooled output (usually the `[CLS]` token's final hidden state) is passed through a small classification or regression head that outputs one score — often normalized to a 0-1 relevance range, though some models output raw logits.

Because the representation is produced jointly, there is nothing to pre-compute: scoring `N` documents against a query requires `N` full forward passes through the transformer.

2.3 Side-by-Side Comparison

Aspect	Bi-Encoder	Cross-Encoder
Input to the model	Query and document, separately	Query and document, concatenated
Attention	Only within each text	Full attention across both texts
Output	A vector per text	A single relevance score per pair
Document encoding	Pre-computable, cached offline	Must be recomputed for every query
Speed at scale	Very fast (ANN search over millions)	Slow if applied to every document
Typical accuracy	Good	Higher — often +5 to +15 NDCG@10 points
Typical role	First-stage retrieval	Second-stage re-ranking

3. Re-Ranking: Why a Second Stage Is Needed

"Re-ranking" is the practice of taking a shortlist of candidate documents — already retrieved by some fast, approximate method — and re-scoring them with a slower, more accurate model so that the final ordering presented to the user (or downstream LLM) is as relevant as possible. Cross-encoders are the standard tool for this second stage precisely because they cannot be used efficiently as the *first* stage: running a full cross-encoder forward pass against every document in a multi-million-document corpus for every query would be far too slow for interactive use.

3.1 The Two-Stage Pipeline

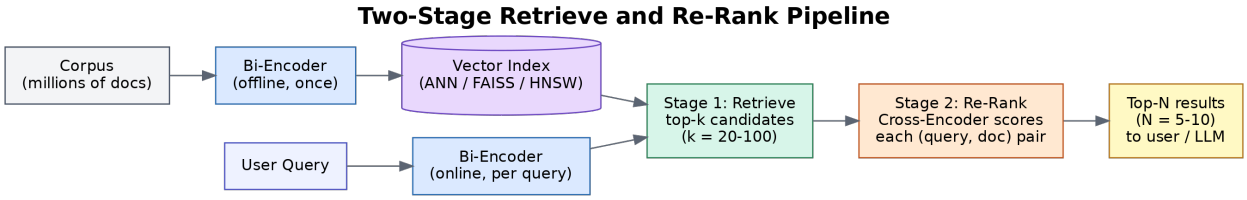

Figure 3: Documents are embedded offline by a bi-encoder and indexed; at query time, the bi-encoder retrieves a shortlist, and the cross-encoder re-ranks it.

Stage 1 — Retrieve. A bi-encoder (or a classical method like BM25, or a hybrid of both) retrieves the top-k candidates from the full corpus, where k is typically in the range of 20 to 100. This stage optimizes for *recall*: it just needs to make sure the truly relevant documents are somewhere in the shortlist.

Stage 2 — Re-rank. A cross-encoder scores each of the k candidates against the query individually, and the candidates are re-sorted by that score. This stage optimizes for *precision*: it decides the final order of the top few results (commonly the top 5-10) that a user will actually see, or that will be passed into an LLM's context window in a RAG application.

3.2 A Worked Example

Suppose a user searches "how to reset a forgotten password" against a support-article corpus. A bi-encoder retrieval step might return, among its top 5 candidates:

1. "Account security best practices" (cosine similarity 0.81)
2. "How to reset your password" (cosine similarity 0.79)
3. "Two-factor authentication setup" (cosine similarity 0.77)
4. "Changing your username" (cosine similarity 0.74)
5. "Password reset instructions for admins" (cosine similarity 0.73)

Purely by embedding similarity, the article actually titled "How to reset your password" ranks *second*, because "Account security best practices" happens to sit slightly closer in embedding space. A cross-encoder, evaluating the full text of the query against each candidate jointly, can recognize that document 2 directly answers the query and would typically re-score it to first place — improving the ranking the user actually sees, without having touched the retrieval step at all.

4. Using Cross-Encoders in Python

The `sentence-transformers` library provides a `CrossEncoder` class that wraps pre-trained cross-encoder models from the Hugging Face Hub. All examples below use Python, the standard tooling language for this ecosystem.

4.1 Installation

```
pip install -U sentence-transformers
```

4.2 Scoring Query-Document Pairs Directly

The core method is `predict()`, which takes a list of `[query, document]` pairs and returns a relevance score for each:

```
from sentence_transformers.cross_encoder import CrossEncoder

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L6-v2")

pairs = [
    ["how to reset a forgotten password",
     "How to reset your password: click Forgot"
     " Password on the login screen."],
    ["how to reset a forgotten password",
     "Account security best practices for teams."],
]

scores = model.predict(pairs)
print(scores)
# e.g. array([ 8.42, -3.15], dtype=float32)
```

Higher scores indicate higher predicted relevance. Note that cross-encoders operate on **pairs** — unlike a bi-encoder, there is no meaningful way to call `predict()` on a single piece of text.

4.3 Convenience Method: `rank()`

For the common case of ranking many candidate documents against one query, `CrossEncoder.rank()` handles pairing, scoring, and sorting in one call:

```
from sentence_transformers.cross_encoder import CrossEncoder

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L6-v2")

query = "how to reset a forgotten password"
candidates = [
    "Account security best practices for teams.",
    "How to reset your password: click Forgot"
```

```
    " Password on the login screen.",
    "Two-factor authentication setup guide.",
    "Changing your account username.",
]

results = model.rank(query, candidates, top_k=3)
for hit in results:
    idx = hit["corpus_id"]
    print(f"{hit['score']:.2f}  {candidates[idx]}")
```

4.4 Full Retrieve-and-Rerank Pipeline

A complete pipeline combines a `SentenceTransformer` bi-encoder for fast first-stage retrieval with a `CrossEncoder` for second-stage re-ranking:

```
from sentence_transformers import SentenceTransformer, util
from sentence_transformers.cross_encoder import CrossEncoder

# Stage 0: corpus and models
corpus = [
    "How to reset your password: click Forgot"
    " Password on the login screen.",
    "Account security best practices for teams.",
    "Two-factor authentication setup guide.",
    "Changing your account username.",
    "Password reset instructions for admins.",
]

bi_encoder = SentenceTransformer("all-MiniLM-L6-v2")
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L6-v2")

# Pre-compute corpus embeddings once (offline, in a real system
# these would be stored in a vector database).
corpus_embeddings = bi_encoder.encode(corpus, convert_to_tensor=True)

query = "how to reset a forgotten password"

# Stage 1: retrieve top-k candidates with the bi-encoder.
query_embedding = bi_encoder.encode(query, convert_to_tensor=True)
hits = util.semantic_search(query_embedding, corpus_embeddings, top_k=5)
hits = hits[0]  # hits for our single query

# Stage 2: re-rank the shortlist with the cross-encoder.
pairs = [[query, corpus[hit["corpus_id"]]] for hit in hits]
ce_scores = cross_encoder.predict(pairs)

for hit, score in zip(hits, ce_scores):
```

```
    hit["cross_score"] = score

reranked = sorted(hits, key=lambda h: h["cross_score"], reverse=True)

for hit in reranked:
    doc = corpus[hit["corpus_id"]]
    print(f"{hit['cross_score']:.2f}  {doc}")
```

This mirrors exactly the two-stage pipeline shown in Section 3.1: the bi-encoder narrows five documents down to a shortlist using cosine similarity, and the cross-encoder produces the final, more accurate ordering.

5. Popular Pretrained Cross-Encoder / Reranker Models

Model	Provider	Notes
`cross-encoder/ms-marco-MiniLM-L6-v2`	Hugging Face / sbert	Fast, good quality; a common default.
`cross-encoder/ms-marco-MiniLM-L12-v2`	Hugging Face / sbert	Larger, slightly higher quality, slower.
`cross-encoder/ms-marco-electra-base`	Hugging Face / sbert	Higher quality, noticeably slower.
`BAAI/bge-reranker-v2-m3`	BAAI	Strong multilingual reranker.
`BAAI/bge-reranker-large`	BAAI	High-quality English-centric reranker.
Cohere Rerank (v3.5)	Cohere API	Hosted, no self-managed infrastructure.
Jina Reranker v2	Jina AI	Multilingual, long-context support.

Model choice is generally a trade-off between accuracy, latency, self-hosting effort, and language coverage. Smaller MiniLM-based models are a reasonable default for prototyping; larger or API-hosted rerankers are typically evaluated once a baseline pipeline is working.

6. Evaluating a Reranker

Because a cross-encoder's whole job is to produce a *good ordering*, evaluation focuses on ranking-quality metrics rather than raw classification accuracy:

- **MRR@k (Mean Reciprocal Rank)** — rewards placing the first relevant document as early as possible in the top k results.
- **NDCG@k (Normalized Discounted Cumulative Gain)** — rewards relevant documents appearing near the top, with a graded (not just binary) notion of relevance and a logarithmic discount for lower positions.
- **MAP (Mean Average Precision)** — averages precision at each point a relevant document is found, across all queries.

`sentence-transformers` provides a `CrossEncoderRerankingEvaluator` that computes these metrics directly: it scores every candidate document for a query, sorts by score, and compares the resulting order against known relevance labels.

```
from sentence_transformers.cross_encoder.evaluation import (
    CrossEncoderRerankingEvaluator,
)

evaluator = CrossEncoderRerankingEvaluator(
    samples=eval_samples,  # list of {"query", "positive", "documents"}
    name="support-articles-eval",
)

results = evaluator(model)
print(results)
# e.g. {'support-articles-eval_MAP': 0.83,
#       'support-articles-eval_MRR@10': 0.88,
#       'support-articles-eval_NDCG@10': 0.85}
```

Published benchmarks (MTEB, BEIR) commonly show a re-ranking stage improving NDCG@10 by roughly 5 to 15 points over a bi-encoder-only pipeline — often the difference between a search system that merely returns plausible results and one that reliably surfaces the right answer.

7. Training a Custom Cross-Encoder

Off-the-shelf rerankers (Section 5) work well for general-purpose text, but domain-specific corpora — legal documents, medical literature, internal support tickets — often benefit from fine-tuning a cross-encoder on labeled or weakly-labeled pairs.

7.1 Loss Functions

`sentence-transformers` exposes several loss functions suited to different label formats:

- **`BinaryCrossEntropyLoss`** — for pairs labeled simply as relevant/not relevant (0 or 1).
- **`MarginMSELoss`** — trains the cross-encoder to match score *margins* produced by a stronger teacher model, a common knowledge-distillation setup.
- **`RankNetLoss`** — a pairwise ranking loss that directly optimizes relative ordering between two documents for the same query.

7.2 Minimal Training Example

```
from sentence_transformers.cross_encoder import (
    CrossEncoder,
    CrossEncoderTrainer,
    CrossEncoderTrainingArguments,
)
from sentence_transformers.cross_encoder.losses import (
    BinaryCrossEntropyLoss,
)

model = CrossEncoder("distilroberta-base", num_labels=1)
loss = BinaryCrossEntropyLoss(model)

args = CrossEncoderTrainingArguments(
    output_dir="./ce-finetuned",
    num_train_epochs=1,
    per_device_train_batch_size=16,
)

trainer = CrossEncoderTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,  # query/doc/label triples
    loss=loss,
)
trainer.train()
```

(Dataset preparation — gathering positive and negative query-document pairs — is typically the larger part of this effort; the training loop itself is comparatively simple.)

8. Trade-offs and Best Practices

8.1 Latency vs. Precision

Cross-encoder re-ranking adds a real latency cost: `k` forward passes through a transformer, one per candidate. Keeping `k` small (20-100) and using a compact model (such as a 6-layer MiniLM) keeps this overhead in the tens of milliseconds range on a GPU, but it is rarely free. For applications with strict sub-100ms latency budgets, re-ranking on the critical path may not be viable, and asynchronous or cached re-ranking strategies should be considered instead.

8.2 Re-Ranking Is a Precision Layer, Not a Recall Fix

A cross-encoder can only re-order the candidates it is given — it cannot recover a relevant document that Stage 1 retrieval failed to surface at all. If overall search quality is poor, the first place to look is chunking strategy, retrieval model choice, and index quality, not the reranker.

8.3 Batch and Cache Where Possible

Because `predict()` accepts a list of pairs, batching all `k` candidates for a query into a single call is significantly more efficient than scoring them one at a time. In production systems with repeated or similar queries, caching reranked results for a short time window can also reduce load.

8.4 Choosing `k`

Smaller `k` (e.g., 20) minimizes latency but risks the reranker never seeing a relevant document that ranked just outside the shortlist. Larger `k` (e.g., 100) is safer for recall but proportionally slower. This is typically tuned empirically against a labeled evaluation set using the metrics from Section 6.

9. Summary

A cross-encoder scores a query and a document jointly, letting full self-attention flow between them, which produces far more accurate relevance judgments than comparing two independently-computed embedding vectors. That accuracy comes at the cost of needing a full transformer forward pass per query-document pair, which is why cross-encoders are used as a **re-ranking** step over a small shortlist rather than as the first-pass retrieval mechanism over an entire corpus. The standard architecture — a fast bi-encoder (or lexical method like BM25) for Stage 1 retrieval, followed by a cross-encoder for Stage 2 re-ranking — is the pattern behind most modern search and RAG systems, typically lifting ranking quality by a wide, measurable margin over retrieval alone.

References

1. Sentence-Transformers Documentation. *Cross-Encoders*. https://sbert.net/examples/cross_encoder/applications/README.html
2. Sentence-Transformers Documentation. *Retrieve & Re-Rank*. https://sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html
3. Sentence-Transformers Documentation. *Cross Encoder Evaluation (CrossEncoderRerankingEvaluator)*. https://sbert.net/docs/package_reference/cross_encoder/evaluation.html
4. Sentence-Transformers Documentation. *Cross Encoder Losses (BinaryCrossEntropyLoss, MarginMSELoss, RankNetLoss)*. https://sbert.net/docs/package_reference/cross_encoder/losses.html
5. Sentence-Transformers Documentation. *Cross Encoder Rerankers Training Guide*. https://sbert.net/examples/cross_encoder/training/rerankers/README.html
6. ZeroEntropy Blog. *Bi-Encoders vs Cross-Encoders*. https://zeroentropy.dev/articles/biencoder-vs-crossencoder/
7. Local AI Master. *Reranking & Cross-Encoders for RAG: BGE, Cohere, Jina (2026)*. https://localaimaster.com/blog/reranking-cross-encoders-guide
8. BSWEN Docs. *Best Reranker Models for RAG: Open-Source vs API Comparison (2026)*. https://docs.bswen.com/blog/2026-02-25-best-reranker-models/
9. Towards Data Science. *Advanced RAG Retrieval: Cross-Encoders & Reranking*. https://towardsdatascience.com/advanced-rag-retrieval-cross-encoders-reranking/